



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems
Department of Information Technology
Fall Semester 2026-27

CLOUD COMPUTING - BITE412L

PROJECT REVIEW-1

**Cloud DNA Framework for Autonomous Cloud Behaviour Modeling and
Self-Evolution**

Submitted by

Mengji Dhanush	23BIT0040
Aniruddha Neema	23BIT0282
Abhijit Potty	23BIT0033

Under the Guidance of

DR. PRIYA V
Associate Professor Senior
Department of Information Technology
School of Computer Science Engineering and Information Systems
VIT, Vellore.

Abstract

Modern cloud-native applications experience highly dynamic workloads, making manual configuration management and resource optimization increasingly inefficient. Existing cloud management solutions primarily rely on reactive autoscaling, predefined policies, or isolated optimization strategies that repeatedly solve similar runtime problems without effectively leveraging knowledge gained from previous adaptations. This often results in redundant experimentation, increased operational costs, and slower response to changing system conditions.

This project proposes an **experience-guided autonomous cloud adaptation framework** that continuously monitors application behaviour, detects runtime deviations, safely evaluates multiple candidate configurations, and incrementally builds a knowledge repository of successful adaptations. The proposed architecture consists of four layers: **Observation**, **Decision**, **Experimentation**, and **Action**. Runtime telemetry - including CPU utilization, memory consumption, latency, request throughput, error rate, security events, scaling behaviour, and recovery metrics - is continuously collected to construct behavioural signatures of cloud applications. Upon detecting a significant deviation from baseline behaviour, a trigger classifier identifies the underlying adaptation category, such as performance degradation, resource scaling, security anomalies, or failure recovery. Multiple candidate configurations are then generated and evaluated within isolated sandbox environments using a multi-objective fitness function that considers performance, availability, cost, and resource utilization. A confidence gate ensures that only validated configurations are promoted to production, while successful adaptations are stored within an experience repository to support future decision-making and reduce unnecessary re-optimization.

The framework is designed entirely on the **Microsoft Azure** ecosystem, utilizing **Azure Monitor** and **Application Insights** for telemetry collection, **Azure Event Grid** and **Azure Functions** for event-driven orchestration, **Azure Kubernetes Service (AKS)** for container orchestration and sandbox execution, **Azure Cosmos DB** for maintaining the adaptation knowledge repository, **Azure Machine Learning** for candidate evaluation and scoring (if required), and **Microsoft Entra ID (Azure AD)** for secure authentication and service

integration. By combining continuous monitoring, safe experimentation, and experience-driven adaptation, the proposed framework aims to improve cloud resilience, reduce adaptation time, and enable progressively autonomous cloud management.

Literature Survey

Paper	Method	Dataset	Advantages	Limitations	Research Gaps
TeaStore: A Micro-Service Reference Application for Benchmarking , Modeling and Resource Management Research	Introduces TeaStore , an open-source microservice benchmark for evaluating cloud resource management, autoscaling, and performance optimization. It supports containerized deployment, monitoring, and configurable workloads.	Uses the TeaStore benchmark with synthetic workloads and real-world traces such as FIFA World Cup 1998 and BibSonomy logs. Performance is evaluated using CPU, latency, throughput, and resource utilization metrics.	• Realistic microservice benchmark. • Open-source and easy to deploy. • Supports autoscaling and performance evaluation. • Suitable for cloud resource management research.	• Serves as a benchmark rather than an adaptation framework. • Focuses mainly on reactive resource management. • Does not learn or reuse previous optimization decisions. • Limited to predefined benchmark scenarios.	TeaStore provides an effective platform for evaluating cloud systems but does not perform intelligent, experience-driven adaptation. There is a need for frameworks that continuously learn from runtime behavior, reuse successful adaptation strategies, and autonomously optimize future cloud

					deployments.
AdaptiFlow: An Extensible Framework for Event-Driven Autonomy in Cloud Microservices	Proposes AdaptiFlow , an event-driven framework based on the MAPE-K model. It separates monitoring, event detection, and adaptation actions, allowing microservices to autonomously respond to runtime events like failures, DDoS attacks, and traffic spikes.	Evaluated using the Adaptable TeaStore microservices benchmark running in Docker. Experiments simulate database failures, DDoS attacks, and high-traffic workloads .	• Modular and extensible framework. • Supports decentralized adaptation. • Minimal code changes to existing microservices. • Handles self-healing, self-protection, and self-optimization.	• Relies mainly on rule-based adaptation. • Reactive rather than predictive. • Limited validation on a single benchmark (TeaStore). • No automatic learning from previous adaptations.	The framework adapts to runtime events effectively but does not learn from historical behavior or reuse successful adaptation strategies . Future systems should incorporate AI-driven learning and knowledge reuse to make cloud adaptation more proactive and intelligent.
Virtual Machine Resource Allocation Optimization in Cloud Computing Based on Multiobjective	Proposes a Multi-Objective Genetic Algorithm (MOGANS) and MOGA-D (MOEA/D-based) approach for optimizing virtual machine (VM)	Evaluated using a simulated cloud data center with multiple physical nodes and virtual machines. Experiments are based on Amazon EC2-like	• Optimizes multiple objectives simultaneously. • Improves VM allocation stability. • Reduces migration overhead and energy consumption.	• Focuses only on VM resource allocation. • Uses evolutionary optimization without learning from previous	The proposed approach finds efficient VM allocations but treats each optimization independently . It lacks mechanisms to

Genetic Algorithm	allocation by considering energy consumption, migration cost, execution time, and system stability.	configurations and compare CPU utilization, memory usage, migration count, stability time, and execution cost.	• Faster convergence using MOGA-D.	executions. • Evaluated mainly in simulated environments. • Does not support autonomous decision-making.	store successful configurations, reuse adaptation knowledge, and continuously improve future cloud optimization decisions, creating opportunities for experience-driven autonomous cloud management.
Resource Allocation in Cloud Computing Using Genetic Algorithm and Neural Network	Proposes a hybrid scheduling framework combining Neural Network Task Classification (N2TC) and Genetic Algorithm Task Assignment (GATA) to optimize cloud resource allocation based on execution time, cost, response time, system	Evaluated using the Google Cluster Traces v3 dataset ($\approx 405,894$ task records). Performance is measured using execution time, response time, cost, and resource utilization.	• Combines AI with genetic optimization. • Considers fairness to prevent task starvation. • Improves execution time, response time, and cost. • Scalable for cloud scheduling.	• Focuses mainly on task scheduling rather than overall cloud adaptation. • Requires training of the neural network. • Does not maintain historical adaptation knowledge. • Limited	The framework improves scheduling decisions but does not continuously learn from runtime adaptations or reuse successful optimization strategies. Future cloud systems should incorporate

	efficiency, and fairness.			evaluation on scheduling workloads only.	experience-driven learning and knowledge repositories to enable autonomous, self-improving cloud management.
Hierarchical Multi-Agent Optimization for Resource Allocation in Cloud Computing	Proposes a Hierarchical Multi-Agent Optimization (HMAO) framework combining an improved Genetic Algorithm (GA) and Multi-Agent Optimization (MAO) . GA maximizes resource utilization, while MAO minimizes bandwidth cost through cooperative service agents.	Evaluated using simulated cloud computing workloads consisting of communication tasks (network receiving, capture, tracking, synchronization, decoding). Performance is measured using resource utilization, bandwidth cost, convergence time, and stability.	• Improves resource utilization and bandwidth efficiency. • Uses distributed multi-agent coordination. • Better scalability than traditional GA approaches. • Reduces computational complexity through hierarchical optimization.	• Assumes complete knowledge of tasks beforehand. • Focuses only on resource allocation and bandwidth optimization. • Does not learn from previous optimization results. • Requires predefined optimization objectives.	The framework coordinates multiple agents for resource allocation but lacks a persistent knowledge base to remember successful adaptations and reuse them in future deployments. Future cloud systems should support continuous learning and experience-driven autonomous

					optimization instead of solving each allocation problem independently.
Netflix Canary Analysis (Lindon et al., 2022)	Uses sequential statistical testing (mSPRT-style, always-valid e-values) to continuously compare a canary deployment's metrics against a baseline, letting the system decide regressed / not regressed at any point in the monitoring window rather than waiting for a fixed sample size.	Statistically rigorous, controls false-positive rate even with continuous peeking at the data, proven at Netflix's production scale; doesn't need the evaluator to pre-commit to a fixed test duration.	Built for a single canary vs. a single baseline, one or a few metrics at a time; has no way to rank or choose among several candidates at once; is purely a detection statistic, not a configuration-search or generation mechanism.	No mechanism to generate or compare multiple candidate configurations, it only judges whether A differs from B, not which of N options is best across several domains at once.	The anytime-valid statistical testing approach could be extended from a single-candidate-vs-baseline comparison into a multi-candidate ranking mechanism, allowing several configurations to be statistically compared against each other simultaneously rather than one at a time against a fixed baseline.
Microservices RL Autoscaler	An RL agent observes live cluster state and	Fully autonomous, adapts continuously to	Optimizes essentially one objective (response	No cross-domain optimization	The single-objective reward function

(Abdel Khaleq & Ra, 2023)	learns a policy to adjust Kubernetes HPA replica counts over time, optimizing mainly for response time, replacing static threshold rules.	traffic patterns without manual threshold tuning; shown to outperform static/rule-based HPA in their tests.	time); only controls one lever (replica count), not the whole application configuration; RL policies need training/convergence time and can behave unpredictably on a genuinely novel event type they haven't seen before (like a first-ever flash sale).	(cost, security, recovery ignored), no history library indexed by event type, no concurrent-candidate comparison, it's one continuously-adapting policy, not a tested population.	could be broadened into a multi-domain scoring system covering cost, security, and recovery alongside response time, and the opaque learned policy could be paired with an auditable, explainable scoring mechanism, since autonomous decisions affecting security or recovery posture generally need to be justifiable rather than black-box.
OpAdviser (database tuning)	A transfer-learning configuration advisor for databases: learns the shape of good search spaces from	Converges to good configurations much faster than tuning from scratch; cuts down the number of	Scoped entirely to database knob tuning, a single subsystem, not whole-application, multi-service	Similarity-based history reuse exists, but only inside one narrow config domain, one	The task-similarity-based search-space narrowing could be generalized beyond database

	past, similar tuning tasks (via task similarity), narrows the search space accordingly, and auto-selects a tuning algorithm.	expensive trial-configurations ; reduces manual effort in choosing which optimizer to use.	configuration; single-objective (throughput/latency), not multi-domain; still a sequential trial-and-refine search, not a concurrently-evaluated population.	objective, and one-at-a-time search, never applied to whole-application deployment configs reacting to live production anomalies.	knob tuning to whole-application deployment configurations, and the sequential trial-and-refine search could be replaced with a concurrently-evaluated candidate population to reduce the number of evaluation rounds needed to converge.
Cloud DB Tuning Survey (Zhang & Babar, 2024)	Surveys the DB-tuning literature into three buckets, search-based (heuristic/Bayesian) , ML-based (regression/RL), and hybrid, most following knowledge-extraction on later feature-selection	Gives a clear map of which techniques are mature vs. emerging, useful for avoiding reinventing an already-solved technique.	Purely descriptive, not a new technique; confined to databases; doesn't address whole-application, multi-domain, or event-triggered reconfiguration at all.	The survey's own taxonomy has no category for concurrent live shadow-population evaluation triggered by a runtime anomaly, confirms this combination is largely	The taxonomy could be extended with a new category for runtime-triggered , concurrently-evaluated, multi-domain configuration search, since the current three categories

	followed by search/recommend.			unexplored, at least in the DB-tuning space it covers.	(search-based, ML-based, hybrid) all assume single-objective, database-scoped tuning rather than whole-application, multi-domain reconfiguration.
K8s ML Autoscaling Review (Soth, Thap & Math, 2026)	A 2026 review of ML-augmented HPA/VPA/Cluster Autoscaler/KEDA, RL, GNN, and transformer-forecast-based scaling, plus emerging multi-agent approaches coordinating HPA+VPA jointly.	Very current; explicitly flags multi-dimensional scaling coordination as an open, unsolved problem, directly useful as supporting evidence for our research-gap claim; covers KEDA, which is event-driven like our trigger model.	Even the coordination work stays scoped to scaling levers (replicas, resource limits), not the fuller config space (cache, connection pools, security posture); no shadow/sandbox pre-validation before a scaling decision hits production.	Confirms multi-dimensional coordination is unsolved even within scaling alone, let alone across the five broader domains (performance, cost, security, scaling, recovery) our project spans.	The multi-agent coordination work identified as an open problem could be extended past scaling levers alone (replicas, resource limits) to include cache, connection-pool, and security-posture parameters, and could incorporate a pre-validation shadow-testing step before a scaling decision is applied to

					production rather than applying decisions directly.
Kephart & Chess, Vision of Autonomic Computing, IEEE Computer 2003	Self-* properties	Introduces four self-managing properties (self-configuring, self-healing, self-optimizing, self-protecting) grounded in a biological nervous-system analogy; MAPE loop first defined here. Establishes a unifying vocabulary for autonomous system management; 20+ years of downstream adoption; applicability across heterogeneous IT environments; high-level goal specification decouples admin	Conceptual/visionary, no concrete engineering methodology; assumes static rule-based policies; does not address distributed/decentralised control; pre-cloud, pre-ML constraints.	No formal specification of loop interactions; gap between high-level vision and practical engineering guidelines for modern cloud, IoT, and AI-driven systems.	Formalise MAPE-K with modern notations (e.g., process algebra, model checking); extend the self-* taxonomy to cover LLM-orchestrated and multi-loop architectures.

		intent from implementation.			
Arcaini, Riccobene & Scandurra, Modeling and Analyzing MAPE-K Feedback Loops, SEAMS 2015	ASM / Formal modelling	Uses Abstract State Machines (ASM) and multi-agent ASMs to formally specify single and distributed MAPE-K loops; meta-property verification detects loop interference at design time. First rigorous formal framework; enables design-time verification; ASMETA support; reusable patterns.	High modelling effort; limited to finite discrete-state systems; scalability concerns; no runtime adaptation; tooling remains academic.	No support for probabilistic/stochastic behaviour; gap between design-time model and runtime dynamics; lacks ML-based planning integration.	Extend ASM semantics with probabilistic/timed extensions; add models@runtime; integrate LLM-generated adaptation plans as verifiable ASM transitions.
Weyns, Oh & Raibulet, Analysis of MAPE-K Loop for Cloud, IoT & CPS, ICSOC 2022 Workshop	Comparative analysis	Systematic examination of five real-world MAPE-K implementations across Cloud, IoT, and CPS. Empirical grounding; useful taxonomy; highlights architectural divergences and	Limited to five systems; descriptive; no performance/overhead evaluation; no generalisable design guidelines.	How MAPE-K should be tailored per domain remains underspecified; no framework for selecting loop configuration.	Develop a domain-specific MAPE-K configuration guide; benchmark loop performance; incorporate AI-driven components.

		common patterns.			
Psaier & Dustdar, Survey on Self-Healing Systems, Computing (Springer) 2011	Detect–Diagnose–Repair	Surveys recovery-oriented, self-stabilising, and survivable systems; introduces detect/diagnose/repair vocabulary and healing taxonomy.	Pre-cloud, pre-microservices; reactive bias; assumes human oversight; no ML/LLM-based diagnostic agents.	Proactive healing largely absent; no ML-driven root-cause analysis; weak integration with MAPE-K Analyze/Plan.	Combine predictive ML and anomaly detection; use LLMs for natural-language root-cause explanation; formalise proactive healing in MAPE-K.
Li, Zhang et al., GenAI for Self-Adaptive Systems, arXiv / ACM TAAS 2025	LLM-in-MAPE-K	Systematic review of 219 papers mapping GenAI contributions to MAPE-K phases and Human-on-the-Loop. First comprehensive roadmap.	Survey only; limited empirical validation; LLM non-determinism unresolved; safety, cost, and latency issues remain.	No standard benchmarks for LLMs in MAPE-K; uncertainty propagation; lack of trust/explainability in safety-critical systems.	Build SAS-specific LLM evaluation suites; integrate uncertainty quantification; formal contracts for planner outputs; optimise latency with smaller fine-tuned models.

Project Objectives

1. **Design and implement a multi-dimensional monitoring layer** that continuously tracks performance, cost, security, scaling, and recovery metrics for a live cloud application using Azure Monitor and Application Insights.
2. **Build a statistical trigger-detection mechanism** that learns a rolling behavioral baseline per metric and flags meaningful deviations in near real time.
3. **Develop a history-aware candidate generation engine** that seeds new configuration candidates using both past successful responses to similar events and controlled parameter mutation.
4. **Implement a concurrent shadow-testing sandbox** that deploys multiple candidate configurations simultaneously against mirrored live traffic, using Azure Container Apps
5. **Design a composite, event-weighted fitness function** that scores each candidate across all five behavioral dimensions and selects a winner with a measurable confidence margin.
6. **Build an autonomous, confidence-gated promotion engine** that applies the winning configuration to production without manual intervention, while preserving all evaluated candidates in a persistent, reusable Genome Library (Cosmos DB) for future events.

Novelty Summary

1. Synchronized, readiness-gated concurrent evaluation instead of parallel testing

Simply spinning up N shadow environments at the same time isn't novel and isn't even fair, because different configurations have different cold-start times, a candidate with a smaller instance size looks artificially better just because it became ready first. Our mechanism instead: (a) launches all N candidates together, but (b) doesn't start the fitness-scoring clock at launch time, it starts each candidate's evaluation window only once that candidate's own readiness probe fires, and (c) aligns all N evaluation windows to the slowest-ready candidate's timestamp before beginning the shared traffic replay. This removes cold-start bias from the comparison, which plain "run them in parallel" does not.

2. Deterministic, lock-step traffic fan-out.

Instead of replaying a separately-generated load script against each shadow instance (which introduces random timing/ordering differences between candidates), we capture a single live traffic trace and broadcast it to all N candidates from one shared dispatch queue, preserving identical request order and identical inter-arrival timing across every candidate. This guarantees candidates are compared on byte-identical conditions, not on statistically-different traffic samples.

3. Self-tuning composite fitness weights, not fixed weights.

Rather than hand-picking fixed weights per trigger category (e.g., flash-sale = 70% performance, 30% scaling), the weight vector for each trigger category is itself adjusted over time based on how accurately past promotions of that category performed against their predicted fitness score - the framework corrects its own scoring rubric as it accumulates evidence, rather than relying on a static, manually-tuned formula.

4. Active security probing inside the sandbox, not passive metric-watching.

During the shadow evaluation window, alongside the mirrored legitimate traffic, the sandbox also injects a small set of synthetic adversarial probes (malformed requests, simulated auth-brute-force patterns) at each candidate simultaneously, so the Security Locus is scored from

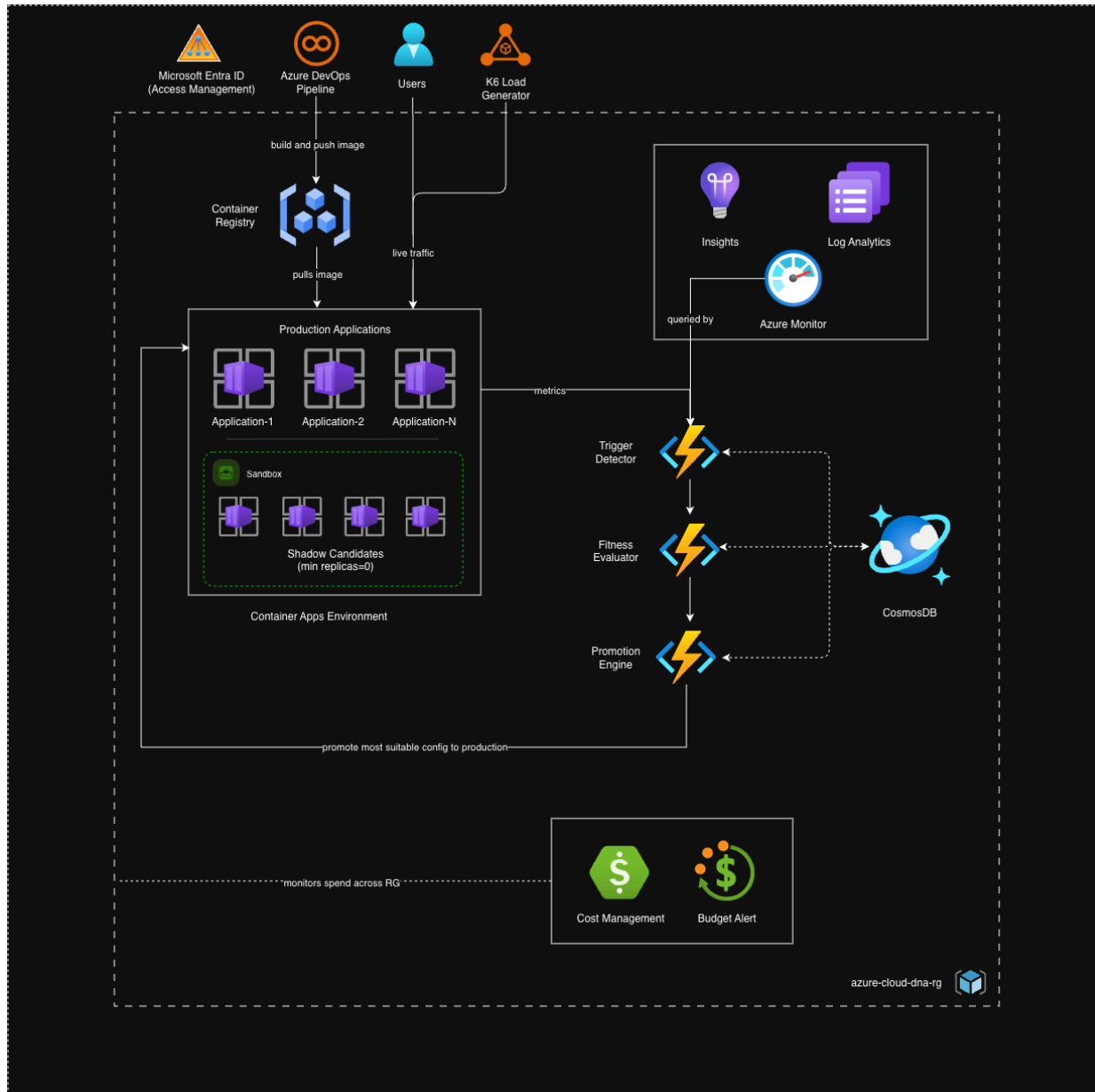
measured, empirical behavior under attack-like conditions, not just from passively-observed metrics.

5. Two-tier autonomy: fast-path vs. slow-path promotion.

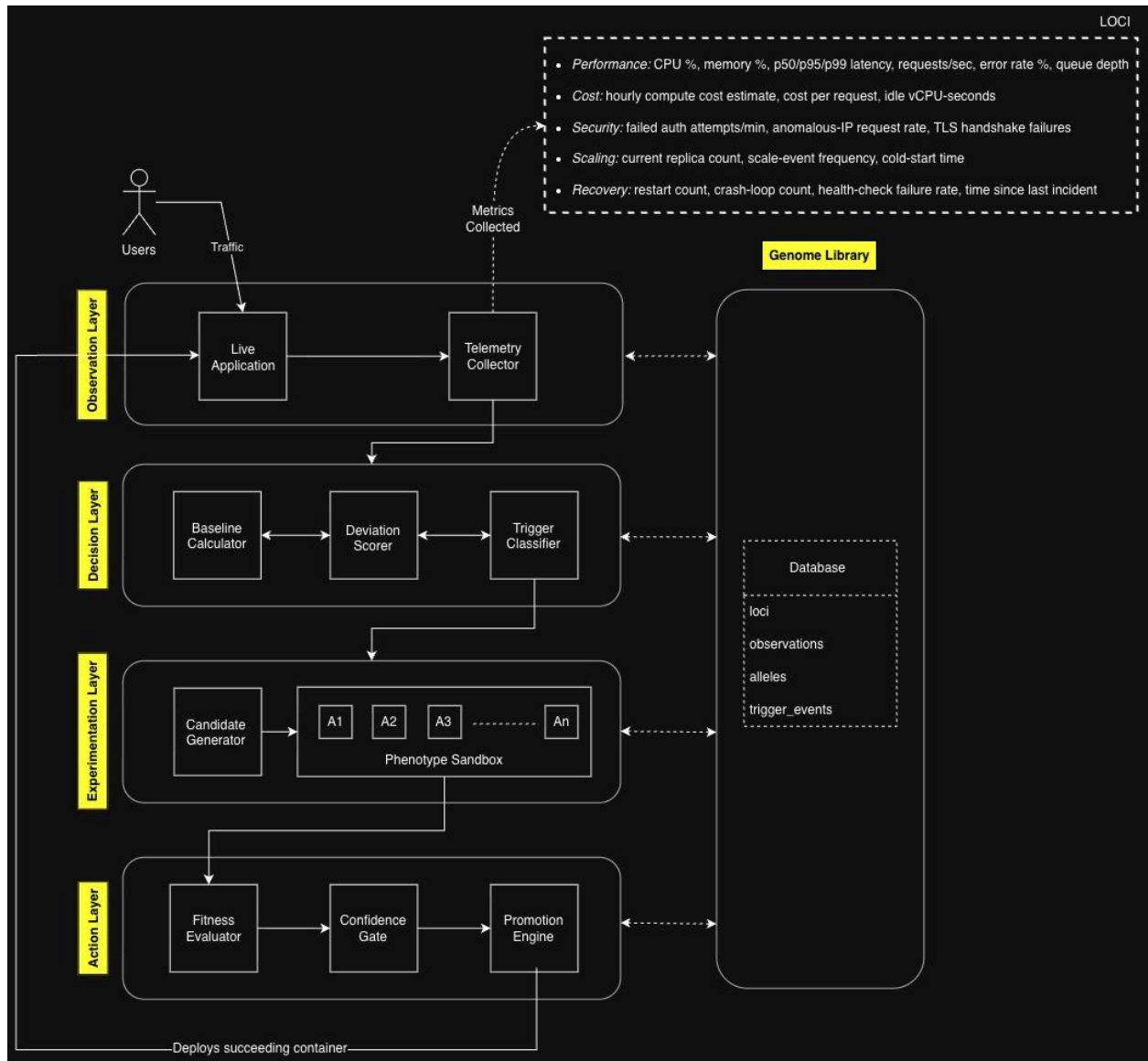
When an incoming trigger's fingerprint is highly similar (above a defined similarity threshold) to a past, successfully-promoted event in the Genome Library, the system can skip the full sandbox cycle entirely and directly re-apply the historically-winning configuration, reserving the full concurrent-candidate experiment for genuinely novel trigger patterns. This turns the framework into a system that gets measurably faster at handling recurring situations over time, which is a concrete, testable claim for your results chapter as well as a patent claim.

Proposed Architecture

i. Cloud Architecture Diagram



ii. Complete System Architecture



Azure Services Planning

Azure Service	Purpose in the Autonomous Cloud Adaptation Framework
Azure Kubernetes Service (AKS)	Host the cloud-native applications (e.g., Medusa) and provide isolated namespaces for candidate evaluation and sandbox experimentation.
Azure Monitor	Continuously collect infrastructure and application metrics such as CPU, memory, latency, throughput, and availability.
Application Insights	Monitor application performance, request traces, dependency calls, exceptions, and user transactions for behavioral analysis.
Azure Functions	Execute event-driven logic including baseline calculation, deviation scoring, trigger classification, candidate generation, fitness evaluation, and promotion workflows.
Azure Event Grid	Trigger autonomous adaptation workflows whenever significant runtime events or anomalies are detected.
Azure Cosmos DB	Store the Adaptation Knowledge Repository containing behavioral signatures, runtime observations, configuration variants, trigger events, evaluation history, and successful adaptations.
Azure Blob Storage	Store telemetry logs, performance reports, experiment outputs, deployment artifacts, and historical datasets used for analysis.
Azure Machine Learning <i>(Future Extension)</i>	Train models for anomaly detection, workload similarity prediction, candidate ranking, and confidence estimation.

Azure AI Foundry / Azure OpenAI (<i>Future Extension</i>)	Generate intelligent configuration recommendations, summarize adaptation reports, or assist administrators with natural language explanations of adaptation decisions.
Azure Load Testing	Evaluate candidate configurations under controlled workloads before deployment to production.
Azure DevOps Pipelines	Automate deployment, validation, rollback, and promotion of successful candidate configurations into the production environment.
Azure Container Registry (ACR)	Store container images for production applications and sandbox candidate deployments.
Microsoft Entra ID (Azure AD)	Secure authentication and role-based access control for administrators, cloud operators, and Azure services.
Azure Virtual Network (VNet)	Provide secure communication between monitoring, experimentation, knowledge repository, and production environments.
Azure Backup	Periodically back up the Adaptation Knowledge Repository, telemetry data, and application state for disaster recovery.
Azure Log Analytics	Aggregate and query telemetry collected from Azure Monitor to support behavioral signature extraction and historical analysis.

Github Repository Link

https://github.com/Ani1702/CloudDNAFramework_Azure_Cloud_Project_2026/

Individual Contribution (until Review-1)

1. Mengi Dhanush:

- Literature Survey of 5 Research Papers (11-15)
- Cloud Architecture Diagram

2. Aniruddha Neema:

- Literature Survey of 5 Research Papers (6-10)
- System Architecture Diagram

3. Abhijit Potty:

- Literature Survey of 5 Research Papers (1-5)
- Novelty Research